RAG systems landscape, late 2026: open source and managed

This note surveys the most advanced retrieval-augmented generation (RAG) systems available in late 2026: open-source projects and managed vendor offerings. For each it covers technique, what makes it advanced, maturity and licence, and where it fits best. It ends with a comparison table and three techniques worth adopting in this repo's grounded-extraction pipeline.

Status: **research note, 2026-10-31.** Every fact links to the primary page it was read on. Star counts, licences and versions are as shown on the GitHub page on **2026-10-31**. The GitHub API was not reachable from this session, so star counts are the rounded figures GitHub shows (for example "36.5k"), and most repo pages showed no release tag. Where a version is missing below, the page did not show one.

Evidence tags:

- **[verified]**: read on the linked primary page on 2026-10-31.
- **[vendor claim]**: a benchmark or quality claim the vendor or authors report about their own system. Not independently checked.
- **[ours]**: a judgement or recommendation made in this note.

---

1. Summary

- **Graph RAG is now standard, and the lighter variants are the ones that get used.** Microsoft GraphRAG (community summaries, global/local/DRIFT search), LightRAG (dual-level graph+vector, 40.5k stars), HippoRAG 2 (Personalized PageRank) and Cognee (graph memory) all build an entity graph at index time. LightRAG has absorbed multimodal parsing (RAG-Anything merged in 2026.05).
- **Agentic retrieval is in every managed offering.** Azure AI Search knowledge bases, Bedrock Managed Knowledge Base and RAGFlow 1.0 all decompose a query into sub-queries, retrieve iteratively and check whether the results suffice. On the research side, Search-R1 trains this loop with reinforcement learning.
- **The retrieval core has settled.** Almost every system uses hybrid lexical+dense search fused with RRF, then a cross-encoder reranker. OpenAI exposes the RRF weights directly. Anthropic's Contextual Retrieval study puts numbers on each layer: 35% / 49% / 67% fewer retrieval failures [vendor claim].
- **Retrieval over page images is mature.** ColPali/ColQwen late interaction over page images (the `colpali-engine` package is now deprecated in favour of Sentence Transformers), Cohere Embed 5 (multimodal, 128k context) and Gemini File Search with `gemini-embedding-2` all index pages as images.
- **"Vectorless" reasoning retrieval is the notable newcomer.** PageIndex (39.9k stars) has an LLM walk a table-of-contents tree instead of using embeddings, and claims 98.7% on FinanceBench [vendor claim].
- **For this repo** [ours]: (1) Contextual Retrieval on top of the existing BM25+embedding hybrid,
    2. a structure-aware or tree-navigation step for long annual reports, PageIndex-style, (3) a second, model-independent groundedness check (HHEM-style NLI) beside `grounding.py`'s quote matching. Claude's native Citations API does not fit as-is, because it cannot be combined with structured outputs (§4.2).

2. Technique vocabulary

Technique	What it is	Where seen
Hybrid search + RRF	BM25/keyword and dense vectors run in parallel, merged by reciprocal rank fusion	R2R, OpenAI, Azure, Kotaemon
Cross-encoder rerank	A second model scores (query, chunk) pairs on a short list	Cohere Rerank 5, Contextual AI, LightRAG, Bedrock
Graph RAG	An LLM extracts entities and relations at index time; retrieval walks or summarises the graph	GraphRAG, LightRAG, HippoRAG, Cognee, Bedrock+Neptune
Late interaction	Token- or patch-level vectors scored with MaxSim	ColBERTv2/PLAID, ColPali/ColQwen
Agentic retrieval	An LLM plans sub-queries, retrieves in a loop and judges whether the results suffice	Azure, Bedrock Managed KB, RAGFlow, R2R, Search-R1
Contextual chunks	Each chunk gets an LLM-written summary of where it sits in the document before indexing	Anthropic Contextual Retrieval
Reasoning/tree retrieval	An LLM navigates a document's heading tree instead of using vector search	PageIndex, RAGFlow "Knowledge Compilation"
Groundedness scoring	An NLI-style model checks that the answer is supported by its evidence	Vectara HHEM, Contextual AI

3. Open-source projects

3.1 Microsoft GraphRAG

- **Technique.** The corpus is split into TextUnits. An LLM extracts entities, relationships and claims, and the graph is clustered hierarchically with Leiden. Community summaries are built bottom-up. There are four query modes: Global (over the community summaries), Local (fanning out from entities), DRIFT (Local plus community context) and Basic (top-k vectors). [verified] https://microsoft.github.io/graphrag/
- **Why it is advanced.** It answers "global" sense-making questions over a whole corpus, which top-k chunk retrieval cannot. [ours]
- **Maturity.** MIT, 36.5k stars, no release tag shown on the repo page (2026-10-31). https://github.com/microsoft/graphrag
- **Best fit.** Thematic questions across large static corpora. Indexing is expensive, because every chunk goes through the LLM. [ours]

3.2 LightRAG (HKUDS)

- **Technique.** A dual-layer graph+vector index with five query modes: local, global, hybrid, naive and mix (the default). A reranker has been the default for mix queries since 2025.08. RAG-Anything was merged in 2026.05 (multimodal parsing via MinerU and Docling), along with four chunking strategies and per-role LLM configuration. OpenSearch storage arrived in 2026.03. [verified] https://github.com/HKUDS/LightRAG

- **Maturity.** MIT, 40.5k stars (2026-10-31), no release tag shown.
- **Best fit.** Graph RAG at much lower indexing cost than GraphRAG, with incremental updates. [ours]

3.3 RAGFlow (InfiniFlow)

- **Technique.** DeepDoc layout analysis, OCR and table recognition (CPU inference in the 1.0 open-source build), template-based explainable chunking, and "Knowledge Compilation" (2026-08-19), which turns documents into wikis, graphs, trees, PageIndex, mind maps and timelines. Agentic RAG has four thinking modes (Low/Medium/High/Ultra). [verified] https://github.com/infiniflow/ragflow
- **Maturity.** Apache-2.0, 93.2k stars (2026-10-31). The news section lists 1.0.0-rc1 on 2026-09-29, and the Docker instructions reference v1.0.0-rc3.
- **Best fit.** A self-hosted, full-stack RAG product with a UI, for complex PDFs. [ours]

3.4 HippoRAG / HippoRAG 2 (OSU NLP)

- **Technique.** "RAG + Knowledge Graphs + Personalized PageRank": a knowledge graph of passages and entities, ranked by PPR from the entities in the query, which models associative, multi-hop memory. HippoRAG 2: arXiv 2502.14802. [verified] https://github.com/OSU-NLP-Group/HippoRAG · https://arxiv.org/abs/2502.14802
- **Maturity.** MIT, 4.1k stars (2026-10-31). A research codebase.
- **Best fit.** Multi-hop QA research and long-term memory. [ours]

3.5 R2R (SciPhi)

- **Technique.** Hybrid semantic+keyword search with RRF, automatic entity and relation graphs, an agentic reasoning agent, and a Deep Research API over the knowledge base and/or the web. [verified] https://github.com/SciPhi-AI/R2R
- **Maturity.** MIT, 8.1k stars (2026-10-31), no release tag shown.
- **Best fit.** A RAG backend you run behind a REST API. [ours]

3.6 Cognee

- **Technique.** Knowledge-graph memory: text becomes entities, relations and chunks, and code becomes a symbol and dependency graph. Retrieval picks graph, vector or code context. Lessons from a session can be distilled into permanent memory (`improve`). [verified] https://github.com/topoteretes/cognee
- **Maturity.** Apache-2.0, 32.9k stars, v1.6.1 (2026-09-24), as seen 2026-10-31.
- **Best fit.** Persistent agent memory. [ours]

3.7 LlamaIndex

- **Technique.** A framework: connectors, indices and graphs, Workflows, and an agent builder. The company's main focus is now LlamaParse (agentic OCR and parsing for 130+ formats, plus extraction). The README says the OSS framework "is no longer the company's primary focus". [verified] https://github.com/run-llama/llama_index
- **Maturity.** MIT, 52.8k stars (2026-10-31). This repo already uses it (`llama_index.retrievers.bm25` in `backend/arp/retrieval/index_cache.py`).
- **Best fit.** Gluing retrievers together, and parsing. [ours]

3.8 Haystack (deepset)

- **Technique.** Modular pipelines with loops and branches (sync, async and streaming), and agents with lifecycle hooks (`before_llm` , `before_tool` , `on_exit`) and concurrent tool calls. [verified] https://github.com/deepset-ai/haystack
- **Maturity.** Apache-2.0, 26.9k stars (2026-10-31).
- **Best fit.** Production pipelines that need explicit control flow. [ours]

3.9 DSPy (Stanford)

- **Technique.** A programming model that optimises prompts and weights for LM pipelines, RAG included. The repo lists the GEPA paper (reflective prompt evolution, July 2025). [verified] https://github.com/stanfordnlp/dspy
- **Maturity.** MIT, 38.9k stars (2026-10-31).
- **Best fit.** Tuning extraction and retrieval prompts against a labelled set, such as a golden set. [ours]

3.10 ColPali / ColQwen (visual retrieval)

- **Technique.** A vision-language model embeds page-image patches into multi-vectors, scored against query tokens with ColBERT-style late interaction. This skips the OCR and layout pipeline. Models: ColPali v1.1–1.3 (PaliGemma-3B), ColQwen2, ColQwen2.5 (Qwen2.5-VL-3B), ColSmol 256M/500M. Paper: arXiv 2407.01449. [verified] https://github.com/illuin-tech/colpali
- **Maturity.** MIT, 2.8k stars (2026-10-31). The README says `colpali-engine` “is now deprecated” and points new projects to Sentence Transformers.
- **Best fit.** Charts, tables and scanned pages, where text extraction loses structure. [ours]

3.11 Kotaemon (Cinnamon)

- **Technique.** A RAG UI with full-text+vector hybrid search, reranking, GraphRAG options (nano-GraphRAG, LightRAG, MS GraphRAG) and citations highlighted in an in-browser PDF viewer. [verified] https://github.com/Cinnamon/kotaemon
- **Maturity.** Apache-2.0, about 25.8k stars (2026-10-31).
- **Best fit.** Document QA with a human-checkable citation UX. [ours]

3.12 txtai (NeuML)

- **Technique.** An “embeddings database” that combines sparse and dense indexes, graph networks and relational storage. It adds LLM pipelines, workflows, smolagents-based agents and an MCP API. The page links “What’s new in txtai 9.0”. [verified] https://github.com/neuml/txtai
- **Maturity.** Apache-2.0, 13.1k stars (2026-10-31).
- **Best fit.** An all-in-one, lightweight semantic search engine in Python. [ours]

3.13 Agentic / deep-research RAG: Search-R1

- **Technique.** An RL framework (on veRL; PPO, GRPO, REINFORCE) that trains LLMs to interleave reasoning with search calls. Papers: arXiv 2503.09516, plus the follow-up study 2505.15117. [verified] https://github.com/PeterGriffinJin/Search-R1 · https://arxiv.org/abs/2503.09516
- **Maturity.** Apache-2.0, about 5.6k stars (2026-10-31). Research.
- **Best fit.** Training your own small search agent. Not relevant when Claude is used through the API. [ours]

3.14 Late interaction: ColBERTv2 / PLAID

- **Technique.** Token-level passage matrices scored with MaxSim. ColBERTv2 adds residual compression (arXiv 2112.01488), and PLAID is the efficient search engine for it (arXiv 2205.09707). [verified] https://github.com/stanford-futuredata/ColBERT
- **Maturity.** MIT, 3.9k stars (2026-10-31).
- **Best fit.** High-precision first-stage retrieval that is cheaper than a cross-encoder. [ours]

3.15 Newer: PageIndex (VectifyAI)

- **Technique.** "Reasoning-based RAG" with no vectors and no chunking: it builds a hierarchical tree index per document, and an LLM reasons down the tree to the relevant sections. It claims 98.7% on FinanceBench against about 50% for vector RAG [vendor claim]. [verified] https://github.com/VectifyAI/PageIndex
- **Maturity.** MIT, 39.9k stars (2026-10-31).
- **Best fit.** Long, well-structured financial and regulatory reports. This is the closest match to this repo's inputs. [ours]
- **Deep dive.** Code walkthrough, the exact FinanceBench setup, costs and a fit proposal are in Appendix A.

---

4. Commercial / managed offerings

4.1 OpenAI file search (Responses API) and vector stores

- A hosted `file_search` tool over vector stores, with "semantic and keyword search", `file_citation` annotations and attribute `filters`. [verified] https://developers.openai.com/api/docs/guides/tools-file-search
- Retrieval guide: RRF hybrid with `embedding_weight` / `text_weight`, a `ranker` and a `score_threshold`, and `rewrite_query`. Default chunks are 800 tokens with 400 overlap (allowed range 100–4096). Search returns up to 50 results. Storage is free for the first 1 GB, then $0.10/GB/day. [verified] https://developers.openai.com/api/docs/guides/retrieval
- **Best fit.** The quickest managed RAG for an OpenAI-model app. [ours]

4.2 Anthropic: Citations and Contextual Retrieval

- **Citations.** Claude returns `char_location` (plain text), `page_location` (PDF) or `content_block_location` (custom content) citations, and also cites `search_result` blocks. `cited_text` does not count toward output tokens. **Citations and structured outputs are incompatible:** combining them returns a 400 error. [verified] https://platform.claude.com/docs/en/build-with-claude/citations
- **Contextual Retrieval.** Claude writes a 50–100-token context for each chunk, which is prepended before embedding and before BM25 indexing. Measured failure rate for top-20 retrieval: 5.7% baseline, 3.7% with contextual embeddings (−35%), 2.9% adding contextual BM25 (−49%), and 1.9% adding a reranker (−67%; 150 candidates reranked down to 20). The one-time cost is about $1.02 per million document tokens with prompt caching [vendor claim]. [verified] https://www.anthropic.com/news/contextual-retrieval
- **Best fit.** Any Claude-based pipeline. Contextual Retrieval works regardless of which vector store you use. [ours]

4.3 Google: Vertex AI RAG Engine and Gemini File Search

- **RAG Engine.** Ingestion, chunking, embedding, corpus indexing, retrieval and generation. Vector backends: RagManagedDb, Vector Search 2.0, Feature Store, Weaviate, Pinecone and others. Parsing: the Document AI layout parser or an LLM parser. A reranking guide is linked. GA varies by region: GA in europe-west3/4, and allowlist or preview elsewhere. [verified] https://docs.cloud.google.com/vertex-ai/generative-ai/docs/rag-engine/rag-overview
- **Gemini File Search.** A managed semantic store with configurable chunking (`chunking_config`), citations carrying the file and page number, metadata filters, and multimodal indexing with `gemini-embedding-2` . Indexing is billed at embedding rates; storage and query embeddings are free. It cannot be combined with Google Search grounding in the same request. [verified] https://ai.google.dev/gemini-api/docs/file-search
- **Best fit.** GCP shops (RAG Engine), or the cheapest hosted RAG for Gemini apps (File Search). [ours]

4.4 AWS Bedrock Knowledge Bases

- There are two kinds. **Managed** KB: AWS-managed embedding and reranking, Smart Parsing per document type (PDF, PPTX, scans, audio, video), agentic retrieval (multi-hop, sub-queries, sufficiency checks), ACL filtering, and MCP via AgentCore Gateway. **Customer-managed** KB: your own vector store (OpenSearch Serverless, Aurora, Neptune). Other options include Neptune Analytics graphs (GraphRAG), structured data stores, Kendra GenAI indexes, multimodal image queries, rerankers and citations. [verified] https://docs.aws.amazon.com/bedrock/latest/userguide/knowledge-base.html
- **Best fit.** AWS enterprises that need connectors and ACLs. [ours]

4.5 Azure AI Search agentic retrieval (Foundry IQ)

- A knowledge base plans sub-queries with an LLM (at `low` / `medium` reasoning effort), runs them in parallel as keyword, vector or hybrid search, semantically reranks each one, and merges the results with references and an activity log. GA REST API `2026-04-01` covers minimal effort; `2026-08-01-preview` adds query planning, answer synthesis and multi-turn. Billing is token-based. [verified] https://learn.microsoft.com/en-us/azure/search/agentic-retrieval-overview
- **Best fit.** Microsoft and Foundry agent stacks. [ours]

4.6 Cohere Rerank 5 and Embed 5

- **Rerank 5:** `rerank-v5.0-pro` and `-fast` , one multilingual model (100+ languages), and YAML for structured documents. [verified] https://docs.cohere.com/docs/rerank-overview
- **Embed 5:** `embed-v5.0-pro` and `-fast` in a shared space, with text, image and mixed input (PDF pages), 128k context, Matryoshka dimensions from 256 to 2048, and float, int8 or binary output. [verified] https://docs.cohere.com/docs/cohere-embed
- **Best fit.** A managed reranker or embedder you can drop into any pipeline, including German text. [ours]

4.7 Vectara

- An “API-first Agentic Platform” with multimodal retrieval, hallucination evaluation and correction. [verified] https://docs.vectara.com/docs/
- HHEM: HHEM-2.1-Open (Apache-2.0, no context limit) scores whether a premise supports a hypothesis on a 0–1 scale. HHEM-2.3 is commercial. [verified]

https://huggingface.co/vectara/hallucination_evaluation_model
- **Best fit.** Regulated RAG that needs a groundedness score. [ours]

4.8 Contextual AI

- The docs list a parser, an instruction-following reranker, a grounded language model, groundedness scoring and agents. No reranker version was visible on that page. [verified] https://docs.contextual.ai/
- **Best fit.** Enterprise RAG where groundedness is the selling point. [ours]

4.9 Glean

- The vendor describes an Enterprise Graph (people, content, activity, synced permissions), an agentic engine that chooses between fast search and a multi-step plan, permissions enforced before the LLM, and semantic+keyword+graph ranking with cited answers [vendor claim]. https://www.glean.com/blog/agentic-reasoning-future-ai · https://www.glean.com/blog/from-enterprise-search-to-enterprise-context-what-ai-agents-actually-need
- **Best fit.** Workplace search across SaaS tools, not document extraction. [ours]

4.10 Pinecone Assistant

- Managed chunking, embedding and storage, answers with file references, a Context API for your own LLM, and an MCP server. [verified] https://docs.pinecone.io/guides/assistant/overview
- **Best fit.** Teams already on Pinecone. [ours]

---

5. Comparison table

System	Graph	Hybrid	Rerank	Agentic loop	Multimodal	Licence / model	Stars (2026-10-31)
MS GraphRAG	✓ (Leiden communities)	–	–	DRIFT	–	MIT	36.5k
LightRAG	✓	✓ (mix)	✓ default	–	✓ (RAG-Anything)	MIT	40.5k
RAGFlow	✓ (compilation)	✓	✓	✓ 4 modes	✓ DeepDoc	Apache-2.0	93.2k
HippoRAG 2	✓ PPR	–	–	–	–	MIT	4.1k
R2R	✓	✓ RRF	–	✓ deep research	–	MIT	8.1k
Cognee	✓ memory	✓	–	–	–	Apache-2.0	32.9k
LlamaIndex	✓	✓	✓	✓ workflows	via LlamaParse	MIT	52.8k
Haystack	–	✓	✓	✓ agents	–	Apache-2.0	26.9k
DSPy	–	–	–	optimiser	–	MIT	38.9k
ColPali/ColQwen	–	–	late interaction	–	✓ page images	MIT	2.8k
Kotaemon	✓	✓	✓	–	PDF highlight	Apache-2.0	25.8k

System	Graph	Hybrid	Rerank	Agentic loop	Multimodal	Licence / model	Stars (2026-10-31)
txtai	✓	✓ sparse+dense	–	✓	✓	Apache-2.0	13.1k
Search-R1	–	–	–	✓ RL-trained	–	Apache-2.0	5.6k
ColBERT/PLAID	–	–	late interaction	–	–	MIT	3.9k
PageIndex	tree	no vectors	–	✓ LLM tree walk	–	MIT	39.9k
OpenAI file search	–	✓ RRF weights	✓ ranker	model-driven	–	managed, $0.10/GB/day >1 GB	–
Anthropic	–	contextual BM25+emb	recommended	–	PDF citations	API feature	–
Vertex RAG Engine / Gemini FS	–	– / semantic	✓	–	✓ gemini-embedding-2	managed	–
Bedrock KB	✓ Neptune	✓	✓	✓ Managed KB	✓ Smart Parsing	managed	–
Azure agentic retrieval	–	✓	✓ semantic	✓ query planning	–	managed, token billing	–
Cohere Rerank 5 / Embed 5	–	–	✓	–	✓ Embed 5	managed	–
Vectara	–	✓	✓	✓	✓	managed; HHEM-2.1 open	–
Contextual AI	–	–	✓ instruction-following	✓	parser	managed	–
Glean	✓ enterprise graph	✓	✓	✓	–	managed	–
Pinecone Assistant	–	–	–	–	–	managed	–

"–" means not stated on the page read, not proven absent.

6. Relevance to this repo

What exists now (skimmed 2026-10-31):

- Evidence retrieval is BM25 by default (`llama_index` BM25Retriever, `backend/arp/retrieval/index_cache.py`), with an optional hybrid mode using BM25 plus local multilingual embeddings (`Settings.hybrid_retrieval_enabled`, `backend/arp/extraction/steps.py`) and an optional local cross-encoder rerank that is off by default because of CPU cost (`backend/arp/retrieval/rerank.py`). German keyword expansion lives in `backend/arp/retrieval/keywords_de.py`.
- Grounding is post-hoc: `backend/arp/grounding.py` normalises text, finds each quoted citation in the source by exact or fuzzy match (threshold 0.92, numbers need a label below 20 characters),

and maps the match to a page, sheet and table. `backend/arp/extraction/vision_check.py` checks values against page images.

Recommendations [ours]:

1. **Contextual Retrieval on the existing hybrid index.** Prepend a Claude-written, prompt-cached context line to each chunk before both BM25 and embedding indexing. It fits the current architecture directly (both indexes already exist), and it is the only technique here with a published ablation on the same BM25+embedding+rerank stack: −49% failures without a reranker, which matters because the rerank step is off by default. Corporate reports suffer from the problem it fixes: a chunk saying "Scope 1 fell 12%" without naming the year or entity. Keep the context out of `grounding.py`'s source text so quotes are still matched against the original document.
2. **Structure-aware retrieval for long reports (PageIndex-style tree navigation).** Annual and sustainability reports have reliable tables of contents and section headings. Have Claude pick sections from the heading tree first, then run BM25 inside them. This cuts recall misses where the keywords are unusual. Treat PageIndex's FinanceBench number as unverified and measure on the golden set before adopting it.
3. **A second, NLI-style groundedness check** (HHEM-2.1-Open, Apache-2.0, or a Claude judge) after `grounding.py`. Quote matching proves a quote *exists*. It does not prove the quote *supports the extracted value* (unit, year, scope). A premise/hypothesis score on (quote, field+value) would feed the existing routing rules (`backend/arp/extraction/routing.py`) as one more quality signal.

Considered and not recommended now:

- **Claude native Citations.** It would give exact char and page spans for free, but it cannot be used together with structured outputs (400 error). Adopting it would mean dropping JSON-schema output on the extraction calls. `grounding.py` already gives the same assurance after the fact.
- **Full graph RAG** (GraphRAG/LightRAG). Extraction asks field-level questions, not global sense-making ones, and the indexing cost does not pay off. Revisit for cross-company questions.
- **ColQwen visual retrieval.** Worth a trial only if the golden set shows misses on chart- or table-only pages. The vision check already covers verification.

`openwiki/` needs no update: this note changes no behaviour.

Appendix A: PageIndex deep dive

Read on 2026-10-31 from a shallow clone of https://github.com/VectifyAI/PageIndex at `2e0f4dd` (2026-10-30, package version 0.2.10 in `pyproject.toml`), the Mafin 2.5 results repo https://github.com/VectifyAI/Mafin2.5-FinanceBench, and the docs at https://docs.pageindex.ai. The code was read, not run. Paths below are relative to the PageIndex repo.

A.1 Indexing, end to end

- **Two local modes.** `submit_document(mode=...)` defaults to `"flash"` locally; `"standard"` is the "full LLM-built tree" (`pageindex/client.py` ~L806). Flash uses the PDF's embedded bookmarks when they look trustworthy (`use_embedded_toc`, default True) and then refines the tree (`optimize="full"`: deterministic merge plus model "expand") (`pageindex/client.py` ~L457). [verified]

- **PDF parsing.** Text-only, per page, with PyPDF2 by default or PyMuPDF (`get_page_tokens`, `pageindex/utils.py` ~L738). Each page becomes `(text, token_count)`. No OCR or layout model in the open source path. [verified]
- **TOC detection (standard mode).** `check_toc` scans the first `toc_check_page_num` pages (default 20, `pageindex/config.yaml`) with one LLM yes/no call per page (`toc_detector_single_page`), extracts the TOC text, and asks the LLM whether page numbers are given (`detect_page_index`) (`pageindex/page_index_classic.py` L176-L295, L857). [verified]
- **Three fallback paths.** `meta_processor` tries `process_toc_with_page_numbers` (map printed page numbers to physical pages via an offset, `calculate_page_offset`), then `process_toc_no_page_numbers` (LLM locates each TOC title in page groups), then `process_no_toc` (LLM *generates* a TOC from ~20k-token page groups with one page overlap, `generate_toc_init` / `generate_toc_continue`, `page_list_to_group_text`). After each path, `verify_toc` has the LLM check that titles really start on the claimed pages; accuracy 1.0 is accepted, >0.6 triggers up to 3 rounds of `fix_incorrect_toc`, otherwise it drops to the next path, and the last path raises `Processing failed` (`pageindex/page_index_classic.py` L1134-L1173). [verified]
- **Large nodes are split recursively.** A leaf spanning more than `max_page_num_each_node` (10) pages *and* at least `max_token_num_each_node` (20,000) tokens is re-indexed with `process_no_toc` over its own pages (`process_large_node_recursively`, L1175). [verified]
- **Page boundaries.** Nodes are page ranges (`start_index` / `end_index`, physical pages), not character spans. `check_title_appearance_in_start` asks the LLM whether a section starts at the top of its page, which decides whether the previous section shares that page; "intro" nodes cover a parent's pages before its first child (`add_intro_nodes`). Sub-page precision is not modelled. [verified]
- **Node JSON.** Output is `{doc_name, [doc_description], structure}`; each node is ordered as `title, node_id, start_index, end_index, key_items, summary, text, nodes` (`format_structure` call in `page_index_main`, L1249-L1300). `node_id` and `summary` are on by default, `text` and `doc_description` off (`pageindex/config.yaml`). [verified]
- **Summaries.** `summarize_tree` runs bottom-up with concurrency 64: a parent's summary is built from its children's summaries plus pages no child covers; small leaves reuse their raw text instead of an LLM call; the prompt asks for "the main points covered in the partial document" (`pageindex/utils.py` L934, L1450). [verified]
- **Prompt-injection hardening.** Document text is wrapped/sanitised (`_wrap_doc_text`, `_secure_doc_text`) and some prompts get a `_SYSTEM_HARDENING` prefix (`page_index_classic.py` L25-L91). [verified]
- **Tree optimizer.** `pageindex/tree_optimize.py` merges or expands nodes to minimise worst-case search cost measured in pages (routing cost 1 page per visited node). This is a deterministic cost model, not learning. [verified]

A.2 Retrieval

- **LLM tool use over the tree, not MCTS.** The README cites AlphaGo as inspiration (README "Inspired by AlphaGo"), but the open-source retrieval is an agent with tools: `get_document_structure()` returns the tree (split into `part`s when too big), and `get_page_content(pages=...)` returns raw pages (`pageindex/agent_tools.py` L191-L239, L910, L1011). The chat model decides which nodes to open. No Monte Carlo search, value function or rollouts appear in the code (grep for `MCTS` / `Monte` returns nothing). [verified]
- **Vectors.** No embeddings in the per-document path. Cloud `browse_documents` offers `sort="relevance"` ("semantic relevance to `query`") across documents (`agent_tools.py` ~L122),

and the Oct '26 Page Search API is a separate hosted retriever ("about a second", README L33) whose method is not in this repo. [verified; Page Search claims are vendor claim]

A.3 Models, cost and latency

- **LLM access via LiteLLM.** Any provider (`"anthropic/claude-sonnet-5-5"` style names); defaults `DEFAULT_INDEX_MODEL = "gpt-6-luna"`, `DEFAULT_CHAT_MODEL = "gpt-6-sol"` (`pageindex/utils.py` L1576). Optional extras for the Claude Agent SDK / Anthropic SDK and `claude-code/` subscription models (flash mode only) (`pyproject.toml` , `page_index_main`). [verified]
- **Indexing cost.** "About $0.001 per page" with `gpt-5.6-luna` ; 9 to 1,098 page PDFs took ~13 s to 4.5 min (README "Local indexing cost and time"). [vendor claim]
- **Query cost.** Native whole-PDF input costs 2.1x more at 52 pages and 16.6x at 420 pages than PageIndex retrieval (README "Cost per query vs. native PDF input"). [vendor claim]
- **Call count (standard mode).** Up to ~20 TOC-detection calls, 1-3 TOC extraction/transform calls, one verification call per TOC entry, fix rounds, one start-of-page check per entry, plus one summary per non-trivial node. For a 300-page report with ~150 headings that is roughly 300-500 calls. [ours, from the code paths above] A third-party estimate gives ~137 calls per document (secondary: https://towardsdatascience.com/proxy-pointer-rag-achieving-vectorless-accuracy-at-vector-rag-scale-and-cost/).

A.4 The 98.7% FinanceBench number [vendor claim]

- **System.** Mafin 2.5, Vectify's commercial financial-QA product built *on* PageIndex, not the open-source library alone (https://github.com/VectifyAI/Mafin2.5-FinanceBench README). [verified]
- **Subset.** The FinanceBench public set; `result_gpt4o.json` and `result_deepseekv3.json` each hold 150 questions (the open-source sample of https://arxiv.org/abs/2311.11944), with all documents in one database. Same 98.7% with GPT-4o and DeepSeek V3 as answer models. [verified]
- **Judge.** An LLM answer-equivalence judge, `gpt-4o-2024-11-20` (`check_answer_equivalence` in `eval.py`), plus vendor-run human re-evaluation of ambiguous or invalid questions (`human_evaluations/*.csv`). So the grader is the vendor's own LLM judge plus the vendor's own annotators. [verified]
- **Baselines.** Competitor numbers (Fintool 98% on 66.7% coverage, Perplexity 45%, ChatGPT-4o+Search 31%) are copied from other sources, not re-run, and the "vector RAG ~50%" bar has no published setup in either repo. [verified that they are not re-run; the figures themselves are vendor claim]

A.5 Limits

- **No TOC:** handled by LLM TOC generation (`process_no_toc`), the most expensive and least reliable path (accuracy-gated fallbacks above). [verified]
- **Scanned / image PDFs:** not handled locally; README's Local vs Cloud table lists "Text-based PDFs" locally and OCR/image understanding as Cloud-only. [verified]
- **Input types:** `page_index_main` raises on non-PDF input; Markdown has a separate `page_index_md.py` . [verified]
- **Many documents:** per-document trees; corpus-level search ("PageIndex File System") is Cloud-only (README "More About PageIndex Cloud"). Secondary reviews advise against it for broad search over many short documents (https://rubinlake.com/en/technology-radar/ai-and-data-engineering/pageindex). [verified / secondary]
- **Latency per query:** several sequential tool calls with a strong model; no published p50. [ours]

A.6 License, hosting, pricing, releases

- MIT, “Copyright (c) 2025 Vectify AI” (`LICENSE`). [verified]
- Cloud: hosted indexing with OCR, storage, API, MCP server (https://docs.pageindex.ai/mcp) (README table). Pricing: $0.01/page one-time indexing, $0.001/active page/month (first 1,000 free), no retrieval fee, LLM inference billed by your own provider, $10 free credits (https://docs.pageindex.ai/pricing). [verified]
- Releases: SDK with local and cloud modes (Aug ’26), Page Search API (Oct ’26) (README updates list); latest commit 2026-10-30 (#622, reasoning/caching fixes for Claude routes). [verified]

A.7 Independent comments (secondary)

- The FinanceBench comparison pits a specialised product against general-purpose LLMs and should be replicated (https://dev.to/andrew-ooo/pageindex-review-vectorless-rag-that-hit-987-accuracy-2fm6).
- Indexing cost grows linearly with corpus size; a hybrid “proxy-pointer” design is proposed as cheaper (https://towardsdatascience.com/proxy-pointer-rag-achieving-vectorless-accuracy-at-vector-rag-scale-and-cost/).
- No peer-reviewed paper or independent benchmark of the open-source library was found. [ours]

A.8 Fit for this repo [ours]

What exists: `field_evidence` (`backend/arp/extraction/field_graph.py` ~L124) chunks routed documents (`chunk_document`, `backend/arp/ingestion/parsing.py`), applies a substring `section_filter` on configured section names (`backend/arp/planning/doc_routing.py`), then ranks with BM25 (optionally hybrid + rerank) and keeps `evidence_max_chunks` (12) per field. `DocumentChunk.section` already carries the heading.

Proposal, smallest version:

1. **Tree without an LLM.** Build a heading tree per document from the PDF outline when present, else from the distinct `chunk.section` values in order, with page ranges. Cache it per `doc_id` beside the span cache. This is PageIndex’s flash idea without its LLM refinement.
2. **One section-selection call per field group, not per field.** Give Claude (a small model, prompt-cached tree) the tree plus the group’s field labels/definitions; it returns `node_id`s per field. Replace the substring `section_filter` with “chunks whose page range overlaps the chosen nodes”, keeping the current fallback (`r.fallback` / `field.high_risk` → all chunks) when the selection is empty.
3. **BM25 unchanged inside the selection.** Grounding stays on original text, so `grounding.py` is untouched. Gate behind a setting (e.g. `section_selection_enabled`, default off) so it enters `input_hash` run settings.

Cost per report: a 300-page report tree with ~200 headings and no summaries is ~4-6k tokens; cached after the first call. With ~10 field groups that is ~10 calls of ~6k input / ~300 output tokens, roughly $0.01-0.05 per report on a Haiku-class model, versus ~$0.30 (vendor $0.001/page) or several hundred calls for a full PageIndex standard-mode build. Adding node summaries would add one call per node and is the first thing to skip.

Testing:

- **Retrieval first, no extraction LLM:** `arp golden-set retrieval` (`backend/arp/golden_set/retrieval.py`) replays `field_evidence` and measures recall of reviewer-approved spans. Run it with the setting off and on; the selection call is the only LLM cost. Adopt only if recall rises without a drop on any field family.
- **Then end to end:** `arp golden-set run` with the setting on; it must keep every case passing (CI gate for `arp/extraction` changes). Record misses where the selected section excluded the gold span.
- Watch for reports whose outline is missing or flat (sections all `None`): the tree falls back to page groups and the step should switch itself off for that document.